Operating Systems Course - Laboratory Project

2026-04-27

1 Instructions

Please read the following instructions before starting the project.

- **Structure**. This project is worth 10 points out of the total grade of the course. As anticipated in the syllabus:
  - 20% of the grade will be based on the code you write: correctness, adherence to the specifications, code quality, etc.
  - 10% of the grade will be based on the report you write: clarity, completeness, presentation, etc.
  - 70% of the grade will be based on the presentation you will give at the exam.
- **Submission.** You **must** submit an archive that:
  - is named
    - `${surname1}_${surname2}_${surname3}.tar.gz` after the surnames of the

    group members. For example, if the surnames of the group members are Rossi, Bianchi and Verdi, the file must be named `Rossi_Bianchi_Verdi.tar.gz`. The order of the surnames does not matter.
  - contains a report, called `report.pdf`, which contains a description of your project, the design choices you made, and any other information you think is relevant. The report must be in PDF format and must be no longer than 5 pages.
  - contains a folder called `code`, which contains the source code of your project, organized as you wish. The code must be written in C and Bash, and it must compile and run correctly on the laboratory computers (or any Ubuntu 24.04 machine).
- **Compilation.** The project must include a way to compile the code and produce the necessary executables to run the project. You can choose to use either:
  - A `Makefile`, which compiles the code and produces the necessary executables to run the project. The Makefile must support at least the following targets:
    - `build`: compiles the project and produces the files required to run the project (check below for the required executables).
    - `clean`: removes all compiled files and executables, and restores the system to a clean state. It must also take care of cleaning up any IPC resources used by the project (e.g., named pipes).
    - `run`: executes one (possibly out of many) scenario of the project, at your choice, using the program(s) that the project requires (e.g., the bootstrapping program, the user script, etc.). Tip: you can choose to use the `ARGS` variable to allow passing arguments to the bootstrapping program when running it (`make run ARGS "--num-cooks=5"`).
  - A Bash script, called `build.sh`, that does exactly the same things stated abpove for the Makefile, but using Bash instead of Make. The script must support at least the following commands:
    - `./build.sh build`: compiles the code and produces the necessary executables to run the project.
    - `./build.sh clean`: removes any compiled files and executables, and restores the system to a clean state.
    - `./build.sh run`: executes a scenario of the project, at your choice, using the program(s) that the project requires.

- **Limitations.**
  - You can freely work on the project with your group and ask for help from the instructors or online, but you must write the code yourself. Plagiarism will be verified using software tools and will be punished according to the university regulations.
  - Any project not written in C and Bash (even C++, even if it compiles and runs correctly) will not be accepted.
  - You can use third-party libraries so long as they are included in your submission and they cause no issues with compilation and execution.
  - Solutions that deviate from the specifications will be penalized depending on the severity of the deviation. If you are unsure about whether a certain design choice is acceptable, please ask the instructors (only after having read the specifications again).
- **Deadline.**
  - You (as a group) can choose to submit the project at any exam attempt of your choice: June, July, September (2026); January, or February (2027). Coordinating with your group members to choose the best time for you is your responsibility.
  - The attempt in which you submit the project will be the attempt in which you will attend the exam, **as a group**. If you submit the project in the June attempt, you will attend the exam in June, and so on.
  - In the Project submission page you can see on Moodle, you will see a deadline. That deadline is a hard deadline for the next exam attempt and is no later than five days before the exam date you see on Esse3. Late submissions will not be possible, as Moodle will automatically close the submission page at the deadline.
  - Once the attempt has passed, the instructors will re-open the submission page for the next attempt, and so on.
  - You can upload, delete and re-upload the project files as many times as you want. Once you are sure that your project is ready for submission, press the "Submit" button in Moodle to submit the project. That action will lock the submission page and make your submission final for that attempt.

2 Project 2026-5 - *Restaurant*

2.1 Setting

In this project, you will implement a simulation of a restaurant using threads. The restaurant will have customers, waiters, and cooks, each represented by a separate thread. The threads will communicate with each other using shared data structures and synchronization primitives to coordinate their actions.

The restaurant will have a menu of dishes, each with specific requirements for kitchen resources (e.g., pots, pans, burners) that cooks need to acquire before they can start cooking. The restaurant will also have a limited number of kitchen resources, and cooks will need to release the resources once they are done cooking.

The performance of the restaurant will be measured by a scoring system that rewards cooks for preparing dishes quickly and penalizes them for their lack of cleanliness and for taking too long, with respect to the customers' patience levels.

2.2 Specification

2.2.1 Menu

The restaurant has a menu of dishes, each with name, price, cooking time and requirements. The menu is loaded from a CSV file following the format:

```
name,price,time,requirements
```

- `name`: the name of the dish (e.g., "Carbonara");
- `price`: the price of the dish in dollars (e.g., 12);

- `time`: the time it takes to cook the dish in minutes (e.g., 15);
- `requirements`: a list of kitchen resources required to cook the dish separated by `;` and with optional quantities specified after a `:` (e.g., "pot;pan;burner:2" means that the dish requires a pot, a pan, and 2 burners).

2.2.2 Kitchen resources

The restaurant has a **limited number** of kitchen resources (e.g., pots, pans, burners) that cooks can use to prepare dishes. The availability of these resources is represented by a shared data structure that keeps track of how many of each resource are currently available. Cooks will need to acquire the necessary resources before they can start cooking a dish, and they will release the resources once they are done.

The list of kitchen resources and their quantities is loaded from a CSV file following the format:

```
resource,quantity,clean_time
```

- `resource`: the name of the kitchen resource (e.g., "burner");
- `quantity`: the total quantity of the resource available in the kitchen (e.g., 4);
- `clean_time`: the time it takes to clean the resource after use in minutes (e.g., 1).

2.2.3 Score

The restaurant score is a **global integer variable** that represents the overall performance of the restaurant. The score updates are triggered by some actions of the cooks and the customers. Cooks earn points for preparing dishes quickly, while they lose points for taking too long or for not cleaning the resources properly. Customers affect the score if they leave the restaurant without being served all their dishes.

2.2.4 Sink

The restaurant has a sink that cooks can use to clean the kitchen resources after they have finished using them. The cleaning process takes a certain amount of time, depending on the resource, and during this time, the resources are not available for use by other cooks. The cleaning of multiple resources **must** be done sequentially at the sink, and the sink can only be used by one cook at a time.

2.2.5 Parties

Each party in the restaurant (customers, waiters, cooks) will be represented by a separate thread. The threads will communicate with each other using shared data structures and synchronization primitives to coordinate their actions.

2.2.5.1 Customers A customer is a thread that is randomly spawned by the main thread and will randomly place an order from the menu.

An order is a list of dishes from the menu that the customer wants to eat. The dishes may be delivered in any order, and as long as the customer receives all the dishes in time, they will be satisfied and contribute positively to the restaurant's score.

The customer will have a patience level that determines how long they are willing to wait for their food before leaving the restaurant. If the customer's patience level is exceeded, they will leave the restaurant without being served. The patience level will be randomly assigned to each customer when they are spawned. The customer's order is determined first; the patience level is then assigned as a random integer strictly greater than the total cooking time of the dishes in the order, to ensure that they have a chance to be served.

In case the customer is served, their contribution will be added to the restaurant's score following this formula:

$$\left\lceil \texttt{total_price} \cdot \left(1 - \frac{\texttt{time_to_serve}}{\texttt{patience_level}}\right) \right\rceil$$

where `total_price` is the sum of the prices of the customer's order, `time_to_serve` is the time it takes for the customer's order to be served, and `patience_level` is the customer's patience level.

On the other hand, if the customer leaves without being served, a penalty will be deducted from the restaurant's score following this formula:

$$\left\lceil \texttt{total_price} \cdot \log_2 \left(1 + \frac{\texttt{patience_level}}{1 + \texttt{number_of_dishes_served}}\right)\right\rceil$$

where `number_of_dishes_served` is the number of dishes that were served to the customer before they left.

The update of the restaurant's score and the customer's patience level done by the customer **must** be carried out in a thread-safe way.

2.2.5.2 Waiters A waiter is a thread that is responsible for taking orders from customers, delivering them to the cooks, and serving the food to customers once it is ready.

Since the cooks are constantly preparing dishes, the waiters decide which orders to deliver to which cooks: given an order, a waiter can assign all of the dishes in the order to a cook, or subdivide the order into multiple parts and assign each part to a different cook. You may use any scheduling algorithm you prefer for this task, but you **must** consider the following factors:

- The cooking time of the dishes in the order;
- The patience level of the customer who placed the order;
- The price of the dishes in the order;
- The current workload of the cooks (i.e., the status of the queues of dishes assigned to each cook).

After a cook finishes preparing a dish, the waiter responsible for that order will be notified and will serve the dish to the customer.

If the waiter is not occupied with other tasks, they **must** entertain waiting customers. Entertainment is implemented as a random event that occurs with a configurable probability, and it can increase or decrease the customer's patience level by a random amount. The update of the customer's patience level **must** be done by the waiter thread in a thread-safe way.

2.2.5.3 Cooks A cook is a thread that is responsible for preparing the dishes ordered by customers.

Each dish is prepared sequentially by a single cook, and the cook can only prepare one dish at a time. To prepare a dish, a cook will need to acquire the necessary kitchen resources as specified in the menu. If the required resources are not available, the cook may decide:

- to stay idle and wait until the resources become available;
- to start preparing another dish that requires different resources;
- to clean some of the resources that are currently dirty (see below).

You are free to implement this decision-making process in any way you prefer, but you **must** ensure that the cook does not acquire resources that are currently being used by other cooks: either they are cooking a dish with them or they are cleaning them.

Once the cook has acquired the necessary resources, they will spend the specified cooking time preparing the dish. After the dish is ready, the cook will release the resources and notify the waiter that the dish is ready to be served.

After a cook finishes preparing a dish, they can choose to clean the kitchen resources they used or to pile them up and leave them dirty (e.g., if they are in a hurry to start cooking another dish). Subsequent cooks that need the same resources and find them dirty, can choose to either clean them or use them as they are. If they choose to use them as they are, they will be able to start cooking immediately, but they will decrease the restaurant's score following this formula:

$$\left\lceil 2^{\texttt{consecutive_dirty_uses}} \cdot \log_2 (1 + \texttt{clean_time})\right\rceil$$

where `consecutive_dirty_uses` is the number of consecutive times the resource has been used without being cleaned, and `clean_time` is the time it takes to clean the resource as specified in the kitchen resources file.

The cleaning process is performed at a sink, which is a shared resource that can only be used by one cook at a time. Cooks that choose to clean a resource will need to acquire the sink and spend the required cleaning time for each resource before they can release them for use by other cooks. The cleaning process takes a certain amount of

time, depending on the resource, and during this time, the resources are not available for use by other cooks. The cleaning of multiple resources **must** be done sequentially at the sink, and the sink can only be used by one cook at a time.

2.2.6 Main thread

The main thread is responsible for initializing the simulation, spawning the customer threads, and managing the overall flow of the restaurant.

The main thread will load the parameters from the environment variables (see below), initialize the shared data structures for the menu, kitchen resources, and restaurant score, and spawn the cook and waiter threads. It will also randomly spawn customer threads at random intervals until the total number of customers specified in the environment variables is reached.

The main thread will also be responsible for printing a lightweight periodic status update at regular intervals. This summary **must** include at minimum:

- the current score,
- the number of customers currently in the restaurant,
- the percentage of spawned customers with respect to the total number of customers specified in the environment variables.

2.2.7 Randomization

All the randomization in the simulation is generated using any random number generator of your choice, but you **must** ensure that the randomization is reproducible by using a fixed seed for the random number generator. This means that if you run the simulation multiple times with the same seed, you should get the same sequence of random events (e.g., customer arrivals, orders, patience levels, entertainment events, etc.).

To do so, each thread that needs to generate random numbers should initialize its own random number generator with a seed that is derived from an output of the main thread's random number generator. For example, the main thread can generate a random seed for each thread and pass it as an argument when spawning the thread.

There are many seedable random number generators that would suit this purpose: here are some examples in C.

2.2.8 Concurrency

- Cooks must safely acquire and release kitchen resources using appropriate synchronization primitives to prevent two cooks from using the same resource simultaneously.
- The sink is a shared resource that can only be used by one cook at a time. Other cooks must block or choose an alternative action (use dirty or work on another dish).
- Customer patience levels can be modified concurrently by the customer thread (decrementing) and by a waiter thread (entertainment). These updates **must** be done in a thread-safe way.
- The global restaurant score is updated by multiple threads (customers leaving, cooks using dirty resources). It **must** be updated in a thread-safe way.
- The main thread spawns customer threads at random intervals. Appropriate synchronization must enforce the `MAX_CUSTOMERS` capacity limit.
- To allow for checking and assessing concurrency, cooking and cleaning operations must use `usleep()` (scaled by `GAME_SPEED`) to simulate time. During this time, other threads must remain able to run (no global locks held during sleep).

2.2.9 Error handling

- All operations in the restaurant system **must** use return codes to indicate the success or failure of the operation. The system **must** define a set of error codes that can be returned by the operations, such as `INVALID_ORDER`, `RESOURCE_UNAVAILABLE`, `INVALID_PARAMETER`, `FILE_NOT_FOUND`, etc. The error codes should be defined and used consistently by both C and Bash code, but do not need to be shared (e.g., you can `#define` the error code in C and use a variable in Bash, as long as they have the same value).
- Any parameter in any Bash and C function that can fail (e.g., due to invalid input, system errors, etc.) **must** handle the error gracefully.
- Any file operation (e.g., reading the menu CSV, loading the resources file, writing log output) **must** check for errors and return appropriate errors if the operation fails (e.g., file not found, malformed CSV).

2.2.10 Edge cases

Check what happens when:

- All kitchen resources are occupied and all cooks are waiting to acquire resources (potential deadlock scenario).
- Two cooks need the same resources at the same time: one should wait or choose an alternative.
- A customer's patience level runs out while their dishes are being cooked: partially served orders must be handled correctly.
- The sink is occupied and a cook wants to clean: they should block or choose a different action, not busy-wait.
- All cooks are cleaning at the sink (only one can, others block): ensure no starvation.
- A customer orders only dishes that require resources already monopolized by other cooks: the order must eventually be fulfilled or the customer leaves.
- The `GAME_SPEED` is set to a very high value (e.g., 100): the simulation should still function correctly and not crash due to timing issues.
- The `.env` file is missing required parameters or has invalid values: the bootstrapping program must reject it gracefully.
- The menu or resources CSV is malformed (e.g., missing columns, non-numeric price): the program must report the error and exit.

2.3 Interfaces

For the sake of testing and grading, the following interfaces **must** be implemented:

2.3.1 Environment variables

All the parameters of the simulation are specified in a `.env` file that is loaded at runtime. The `.env` file contains lines following the format:

```
PARAMETER_NAME=VALUE
```

where `PARAMETER_NAME` is the name of the parameter and `VALUE` is its value. The parameters that you need to implement are:

- `NUM_COOKS`: the number of cook threads in the simulation;
- `NUM_WAITERS`: the number of waiter threads in the simulation;
- `MAX_CUSTOMERS`: the maximum number of customers that can be present in the restaurant at the same time (i.e., the maximum capacity of the restaurant);
- `TOTAL_CUSTOMERS`: the total number of customers that will visit the restaurant during the simulation;

- `MENU_FILE`: the path to the CSV file containing the menu of dishes;
- `RESOURCES_FILE`: the path to the CSV file containing the kitchen resources and their quantities;
- `GAME_SPEED`: a factor that determines the speed of the simulation, where a value of 1 means real-time, a value greater than 1 means faster than real-time, and a value less than 1 means slower than real-time. All time values (cooking times, cleaning times, customer patience levels) must be divided by `GAME_SPEED` when used in `usleep()` calls to scale the simulation speed accordingly.
- `RANDOM_SEED`: the seed for the random number generator to ensure reproducibility of the random events in the simulation.

2.3.2 Bootstrapping program

The project **must** include a bootstrapping program `bootstrap.sh` written in Bash that loads the parameters from the `.env` file and starts the main binary of the project. The bootstrapping program should be responsible for checking the `.env` file and its content before starting the main binary. Mainly it should check that:

- the `.env` file exists and is readable;
- all the required parameters are specified in the `.env` file and have valid values (e.g., positive integers for the number of cooks, waiters, customers, etc.);
- the menu and resources files specified in the `.env` file exist and have the correct format (i.e., they follow the specified CSV format).

If any of these checks fail, the bootstrapping program should print an error message indicating the issue and exit without starting the main binary. If all checks pass, the bootstrapping program should start the main binary of the project (e.g., `./restaurant`). The main binary **must** write its own PID to `/tmp/restaurant.pid` on startup, which is used by `status.sh` to communicate with the running process.

For ease of use, the bootstrapping program can accept command-line arguments to specify a different file with the environment variables (e.g., `./bootstrap.sh --env-file=custom.env`), but if no such argument is provided, it should default to using the `.env` file in the current directory. Additionally, the bootstrapping program can also accept command-line arguments to override the parameters specified in the `.env` file. Following the format:

```
./bootstrap.sh --parameter-name=VALUE
```

where `--parameter-name=VALUE` follows the same format as the parameters in the `.env` file (e.g., `--num-cooks=5`). The bootstrapping program should apply these overrides after checking the `.env` file and before starting the main binary.

2.3.3 Status script

A status script `status.sh` written in Bash must be provided to trigger a full status dump of the running simulation on demand. The script sends `SIGUSR1` to the restaurant process (whose PID is stored in `/tmp/restaurant.pid`). Upon receiving the signal, the process must print a complete snapshot of the restaurant state to its standard output, including:

- the current score,
- the number of customers currently in the restaurant and how many have left unserved,
- the progress through total customers (spawned vs. total),
- the length of the queue of dishes assigned to each cook,
- the current availability of each kitchen resource.

Usage:

```
./status.sh
```